



CHAPTER 6

SETS

Learning Outcomes

Describe the Set data structure as implemented in the Java Collections Framework

Explain the Set data structure implementation of

- ▶ Union
- ▶ Intersection
- ▶ Difference

Apply the concrete classes:

- ▶ HashSet
- ▶ LinkedHashSet
- ▶ TreeSet.

Sets

- ▶ In computer science, a set is an abstract data type that can store unique values, without any particular order.
- ▶ It is a computer implementation of the mathematical concept of a finite set.

Sets

- ▶ The Set data structure is usually used to test whether elements belong to set of values.
- ▶ Sets are used to perform operations on multiple values at once with methods such as
 - ▶ Union
 - ▶ Intersection
 - ▶ Difference.

Sets

- ▶ A set is an efficient data structure for storing and processing non-duplicate elements.
- ▶ You can create a set using one of its three concrete classes:
 - ▶ HashSet
 - ▶ LinkedHashSet
 - ▶ TreeSet.

Java Collections Framework

- ▶ The Set interface extends the Collection interface.
- ▶ It does not introduce new methods or constants, but ensures that an instance of Set contains no duplicate elements.



HashSet

SETS

HashSet

- ▶ The HashSet class is a concrete class that implements Set.
- ▶ You can create an empty hash set using its no-arg constructor or create a hash set from an existing collection.
- ▶ By default, the initial capacity is 16 and the load factor is 0.75.
- ▶ If you know the size of your set, you can specify the initial capacity and load factor in the constructor.
- ▶ Otherwise, use the default setting. The load factor is a value between 0.0 and 1.0.

The HashSet Load Factor

- ▶ The load factor measures how full the set is allowed to be before its capacity is increased.
- ▶ When the number of elements exceeds the product of the capacity and load factor, the capacity is automatically doubled.
- ▶ For example, if the capacity is 16 and load factor is 0.75, the capacity will be doubled to 32 when the size reaches 12 ($16 * 0.75 = 12$).
- ▶ A higher load factor decreases the space costs but increases the search time.
- ▶ Generally, the default load factor 0.75 is a good trade-off between time and space costs.

hashCode

- ▶ A HashSet can be used to store duplicate-free elements.
- ▶ For efficiency, objects added to a hash set need to implement the hashCode method in a manner that properly disperses the hash code.
- ▶ Recall that hashCode is defined in the Object class.
- ▶ The hash codes of two objects must be the same if the two objects are equal.

hashCode

- ▶ Two unequal objects may have the same hash code, but you should implement the hashCode method to avoid too many such cases.
- ▶ Most of the classes in the Java API implement the hashCode method.
- ▶ For example:
 - ▶ The hashCode in the Integer class returns its int value.
 - ▶ The hashCode in the Character class returns the Unicode of the character.
 - ▶ The hashCode in the String class returns $s_0 * 31^{(n-1)} + s_1 * 31^{(n-2)} + \dots + s_{n-1}$, where s_i is `s.charAt(i)`.

```
import java.util.*;

public class TestHashSet {
    public static void main(String[] args) {
        // Create a hash set
        Set<String> set = new HashSet<>();

        // Add strings to the set
        set.add("London");
        set.add("Paris");
        set.add("New York");
        set.add("San Francisco");
        set.add("Beijing");
        set.add("New York");

        System.out.println(set);

        // Display the elements in the hash set
        for (String s: set) {
            System.out.print(s.toUpperCase() + " ");
        }
    }
}
```

[San Francisco, New York, Paris, Beijing, London]

SAN FRANCISCO NEW YORK PARIS BEIJING LONDON

- The strings are added to the set.
- New York is added to the set more than once, but only one string is stored, because a set does not allow duplicates.
- The strings are not stored in the order in which they are inserted into the set.
- There is no particular order for the elements in a hash set.
- To impose an order on them, you need to use the `LinkedHashSet` class.
- Recall that the `Collection` interface extends the `Iterable` interface, so the elements in a set are iterable.
- A `foreach` loop is used to traverse all the elements in the set.

TestMethodsInCollection.java

```
public class TestMethodsInCollection {  
    public static void main(String[] args) {  
        // Create set1  
        java.util.Set<String> set1 = new java.util.HashSet<>();  
  
        // Add strings to set1  
        set1.add("London");  
        set1.add("Paris");  
        set1.add("New York");  
        set1.add("San Francisco");  
        set1.add("Beijing");  
  
        System.out.println("set1 is " + set1);  
        System.out.println(set1.size() + " elements in set1");  
  
        // Delete a string from set1  
        set1.remove("London");  
        System.out.println("\nset1 is " + set1);  
        System.out.println(set1.size() + " elements in set1");  
    }  
}
```

TestMethodsInCollection.java

```
// Create set2
java.util.Set<String> set2 = new java.util.HashSet<>();

// Add strings to set2
set2.add("London");
set2.add("Shanghai");
set2.add("Paris");
System.out.println("\nset2 is " + set2);
System.out.println(set2.size() + " elements in set2");
System.out.println("\nIs Taipei in set2? "
    + set2.contains("Taipei"));

set1.addAll(set2);
System.out.println("\nAfter adding set2 to set1, set1 is "
    + set1);

set1.removeAll(set2);
System.out.println("After removing set2 from set1, set1 is "
    + set1);

set1.retainAll(set2);
System.out.println("After removing common elements in set2 "
    + "from set1, set1 is " + set1);
}
```

TestMethodsInCollection.java - Output

```
set1 is [San Francisco, New York, Paris, Beijing, London]  
5 elements in set1
```

```
set1 is [San Francisco, New York, Paris, Beijing]  
4 elements in set1
```

```
set2 is [Shanghai, Paris, London]  
3 elements in set2
```

```
Is Taipei in set2? false
```

```
After adding set2 to set1, set1 is  
[San Francisco, New York, Shanghai, Paris, Beijing, London]
```

```
After removing set2 from set1, set1 is  
[San Francisco, New York, Beijing]
```

```
After removing common elements in set2 from set1, set1 is []
```

Union

- ▶ In this, we could simply add one Set with other. Since the Set will itself not allow any duplicate entries, we need not take care of the common values.

Intersection

- ▶ We just need to retain the common values from both Sets.

Difference

- ▶ We just need to remove all the values of one Set from the other.



LinkedHashSet

LinkedHashSet

- ▶ LinkedHashSet extends HashSet with a linked-list implementation that supports an ordering of the elements in the set.
- ▶ The elements in a HashSet are not ordered, but the elements in a LinkedHashSet can be retrieved in the order in which they were inserted into the set.
- ▶ A LinkedHashSet can be created by using one of its four constructors.
- ▶ These constructors are similar to the constructors for HashSet.

TestLinkedHashSet.java

```
import java.util.*;

public class TestLinkedHashSet {
    public static void main(String[] args) {
        // Create a hash set
        Set<String> set = new LinkedHashSet<>();

        // Add strings to the set
        set.add("London");
        set.add("Paris");
        set.add("New York");
        set.add("San Francisco");
        set.add("Beijing");
        set.add("New York");

        System.out.println(set);

        // Display the elements in the hash set
        for (Object element: set)
            System.out.print(element.toLowerCase() + " ");
    }
}
```

LinkedHashSet

- ▶ As shown in the output, the strings are stored in the order in which they are inserted.
- ▶ Since LinkedHashSet is a set, it does not store duplicate elements.
- ▶ The LinkedHashSet maintains the order in which the elements are inserted.
- ▶ To impose a different order (e.g., increasing or decreasing order), you can use the TreeSet class.
- ▶ If you don't need to maintain the order in which the elements are inserted, use HashSet, which is more efficient than LinkedHashSet.



TreeSet

SETS

TreeSet

- ▶ **SortedSet** is a subinterface of **Set**, which guarantees that the elements in the set are sorted.
- ▶ Additionally, it provides the methods **first()** and **last()** for returning the first and last elements in the set, and **headSet(toElement)** and **tailSet(fromElement)** for returning a portion of the set whose elements are less than **toElement** and greater than or equal to **fromElement**, respectively.

TreeSet

- ▶ **NavigableSet** extends **SortedSet** to provide navigation methods **lower(e)**, **floor(e)**, **ceiling(e)**, and **higher(e)** that return elements respectively less than, less than or equal, greater than or equal, and greater than a given element and return null if there is no such element.
- ▶ The **pollFirst()** and **pollLast()** methods remove and return the first and last element in the tree set, respectively.

TreeSet

- ▶ TreeSet implements the SortedSet interface.
- ▶ To create a TreeSet, use a constructor.
- ▶ You can add objects into a tree set as long as they can be compared with each other.
- ▶ The elements can be compared in two ways:
 - ▶ using the Comparable interface or
 - ▶ the Comparator interface.

TestTreeSet.java

```
import java.util.*;

public class TestTreeSet {
    public static void main(String[] args) {
        // Create a hash set
        Set<String> set = new HashSet<>();

        // Add strings to the set
        set.add("London");
        set.add("Paris");
        set.add("New York");
        set.add("San Francisco");
        set.add("Beijing");
        set.add("New York");

        TreeSet<String> treeSet = new TreeSet<>(set);
        System.out.println("Sorted tree set: " + treeSet);

        // Use the methods in SortedSet interface
        System.out.println("first(): " + treeSet.first());
        System.out.println("last(): " + treeSet.last());
        System.out.println("headSet(\"New York\"): " +
            treeSet.headSet("New York"));
        System.out.println("tailSet(\"New York\"): " +
            treeSet.tailSet("New York"));
    }
}
```

TREE SET

```
// Use the methods in NavigableSet interface
System.out.println("lower(\"P\"): " + treeSet.lower("P"));
System.out.println("higher(\"P\"): " + treeSet.higher("P"));
System.out.println("floor(\"P\"): " + treeSet.floor("P"));
System.out.println("ceiling(\"P\"): " + treeSet.ceiling("P"));
System.out.println("pollFirst(): " + treeSet.pollFirst());
System.out.println("pollLast(): " + treeSet.pollLast());
System.out.println("New tree set: " + treeSet);
}
```

TestTreeSet.java

- ▶ The example creates a hash set filled with strings, then creates a tree set for the same strings.
- ▶ The strings are sorted in the tree set using the compareTo method in the Comparable interface.
- ▶ The elements in the set are sorted once you create a TreeSet object from a HashSet object using `new TreeSet<String>(set)`.
- ▶ You may rewrite the program to create an instance of TreeSet using its no-arg constructor, and add the strings into the TreeSet object.

TestTreeSet.java

- ▶ `treeSet.first()` returns the first element in `treeSet` (line 20), and `treeSet.last()` returns the last element in `treeSet`.
- ▶ `treeSet.headSet("New York")` returns the elements in `treeSet` before New York.
- ▶ `treeSet.tailSet("New York")` returns the elements in `treeSet` after New York, including New York.

TestTreeSet.java

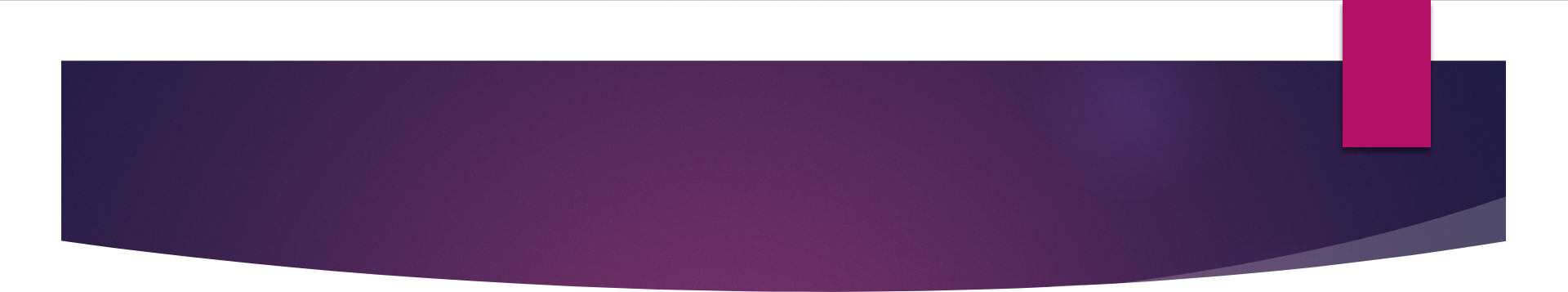
- ▶ `treeSet.lower("P")` returns the largest element less than P in `treeSet`.
- ▶ `treeSet.higher("P")` returns the smallest element greater than P in `treeSet`.
- ▶ `treeSet.floor("P")` returns the largest element less than or equal to P in `treeSet`.
- ▶ `treeSet.ceiling("P")` returns the smallest element greater than or equal to P in `treeSet`.
- ▶ `treeSet.pollFirst()` removes the first element in `treeSet` and returns the removed element.
- ▶ `treeSet.pollLast()` removes the last element in `treeSet` and returns the removed element.

TreeSet

- ▶ All the concrete classes in Java Collections Framework have at least two constructors.
- ▶ One is the no-arg constructor that constructs an empty collection.
- ▶ The other constructs instances from a collection. Thus the TreeSet class has the constructor `TreeSet(Collection c)` for constructing a TreeSet from a collection `c`.
- ▶ In the previous example, `new TreeSet<>(set)` creates an instance of TreeSet from the collection `set`.

HashSet vs TreeSet

- ▶ If you don't need to maintain a sorted set when updating a set, you should use a hash set, because it takes less time to insert and remove elements in a hash set.
- ▶ When you need a sorted set, you can create a tree set from the hash set.

- 
- ▶ If you create a `TreeSet` using its no-arg constructor, the `compareTo` method is used to compare the elements in the set, assuming that the class of the elements implements the `Comparable` interface.
 - ▶ To use a comparator, you have to use the constructor `TreeSet(Comparator comparator)` to create a sorted set that uses the `compare` method in the comparator to order the elements in the set.



END OF CHAPTER 6